Reason. Plan. Act. Physical AI Workshop

Lab 4.2: Intro to AI with deep learning and world models

Almog Anschel and Sarah Keren

The Taub Faculty of Computer Science
Technion - Israel Institute of Technology



Today

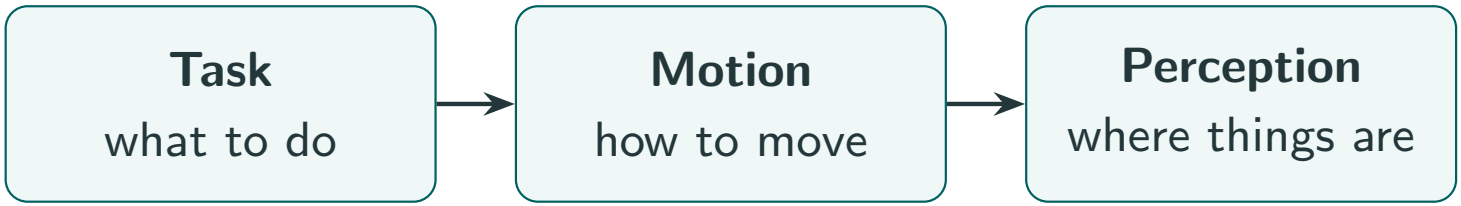
Learn	Classical AI guarantees a plan, given a model; a robot has to build the model
Learn	Move: learning the motion planner from demonstrations
Do	Task 1: copy the demonstrations, and watch the robot fail at a fork
Do	Task 2: train a flow-matching policy that keeps both ways around
Do	Task 3: write the sampler that turns noise into a motion
Learn	Inside a VLA: pixels and a sentence become a motion
Learn	See and act: a VLA on our arm
Do	Task 4: run the VLA on our arm, on a scene it has never seen
Do	Task 5: find the token the target block falls in
Do	Task 6: write attention, see where the robot looks, swap one word
Learn	Conclusion, and your vote
Do	Tasks 7–8: an LLM in the task planner's seat

1.1 Task planning	2.1 Motion planning	3.1 State estimation	4.1 Integrated TAMP
1.2 Inverse kinematics	2.2 Control	3.2 Decoupled TAMP	**4.2 VLAs**

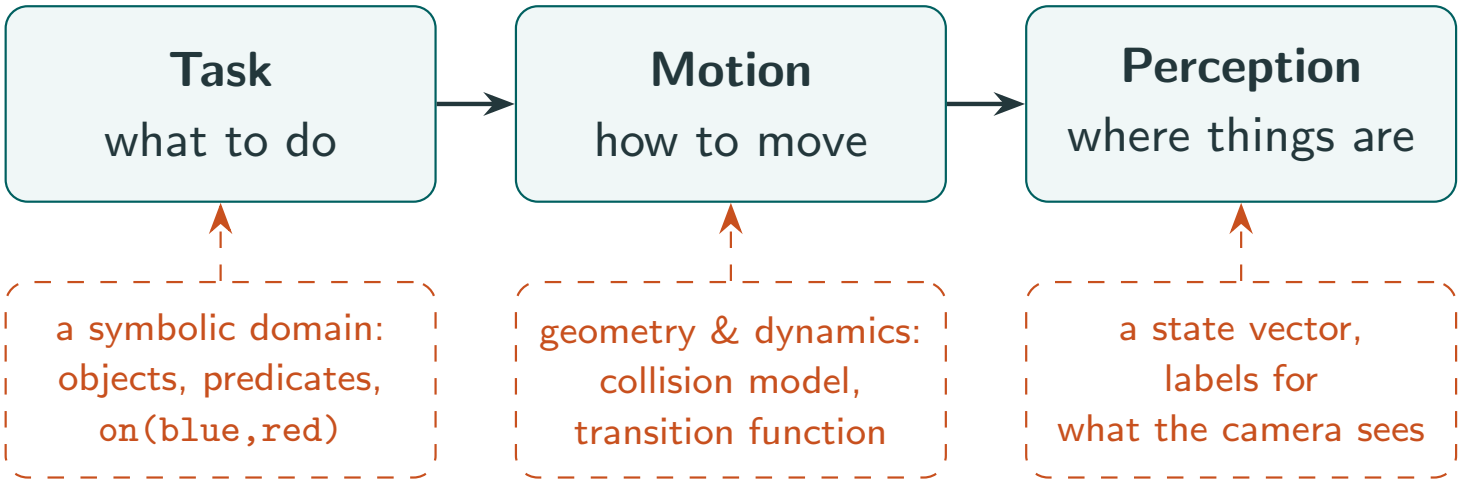


π_0 unloads a dryer · 2× speed · Physical Intelligence, 2024

Each layer assumed a model



Each layer assumed a model



What classical AI takes for granted

Classical AI guarantees a plan, given a model.

A robot has to build that model.

What classical AI takes for granted

Classical AI guarantees a plan, given a model.

A robot has to build that model.

- **Grounding.** Pixels do not arrive labelled `on(blue,red)`.

What classical AI takes for granted

Classical AI guarantees a plan, given a model.

A robot has to build that model.

- **Grounding.** Pixels do not arrive labelled `on(blue,red)`.
- **Dynamics.** Contact, friction, cloth, liquids.

What classical AI takes for granted

Classical AI guarantees a plan, given a model.

A robot has to build that model.

- **Grounding.** Pixels do not arrive labelled `on(blue,red)`.
- **Dynamics.** Contact, friction, cloth, liquids.
- **Specification.** A PDDL domain per task and per object set.

What classical AI takes for granted

Classical AI guarantees a plan, given a model.

A robot has to build that model.

- **Grounding.** Pixels do not arrive labelled `on(blue,red)`.
- **Dynamics.** Contact, friction, cloth, liquids.
- **Specification.** A PDDL domain per task and per object set.

The planner stays. Deep learning learns what it was handed: grounding, dynamics, specification.

Two kinds of AI: which parts to write, which to fit?

Classical: model given

- a person writes S, A, $T(s' \mid s, a)$, the goal
- the algorithm **searches** the model
- output comes with a **guarantee**: valid for the model
- new goal, new object: **replan**, no data
- wrong model: wrong plan

Two kinds of AI: which parts to write, which to fit?

Classical: model given

- a person writes S, A, $T(s' \mid s, a)$, the goal
- the algorithm **searches** the model
- output comes with a **guarantee**: valid for the model
- new goal, new object: **replan**, no data
- wrong model: wrong plan

Learned: model fitted

- a network f_θ: inputs to outputs, nothing in between is named
- **fit** to data: $\min_\theta \mathbb{E} \, \| f_\theta(o) - a^* \|^2$
- output comes with a **distribution**: good where the data was
- new goal, new object: **new data**, retrain
- wrong data: wrong action

Three ways to get it, and what each one costs

	What is learned	What stays classical	Examples	What it costs
1. Learn the model, keep planning	a *world model*: next state given the action	search / MPC on top of it	Dreamer, V-JEPA 2	errors add up; planning in a model that can be wrong

Three ways to get it, and what each one costs

	What is learned	What stays classical	Examples	What it costs
1. Learn the model, keep planning	a *world model*: next state given the action	search / MPC on top of it	Dreamer, V-JEPA 2	errors add up; planning in a model that can be wrong
2. Learn the policy, skip the model	observation → action, directly	nothing	behaviour cloning, Diffusion Policy, **VLA**	demonstrations per robot and task; no check, no guarantee

Three ways to get it, and what each one costs

	What is learned	What stays classical	Examples	What it costs
1. Learn the model, keep planning	a *world model*: next state given the action	search / MPC on top of it	Dreamer, V-JEPA 2	errors add up; planning in a model that can be wrong
2. Learn the policy, skip the model	observation → action, directly	nothing	behaviour cloning, Diffusion Policy, **VLA**	demonstrations per robot and task; no check, no guarantee
3. Hybrid	a proposer: perception → symbols, or an LLM that drafts plans	a planner that *checks* and executes	SayCan, LLM-Modulo	two systems that must agree on the symbols

Three ways to get it, and what each one costs

	What is learned	What stays classical	Examples	What it costs
1. Learn the model, keep planning	a *world model*: next state given the action	search / MPC on top of it	Dreamer, V-JEPA 2	errors add up; planning in a model that can be wrong
2. Learn the policy, skip the model	observation → action, directly	nothing	behaviour cloning, Diffusion Policy, **VLA**	demonstrations per robot and task; no check, no guarantee
3. Hybrid	a proposer: perception → symbols, or an LLM that drafts plans	a planner that *checks* and executes	SayCan, LLM-Modulo	two systems that must agree on the symbols

"World model" here means one thing only: a learned predictor of the next state given the current one and an action. Nothing more.

Inside a VLA: from pixels to a decision

Architecture

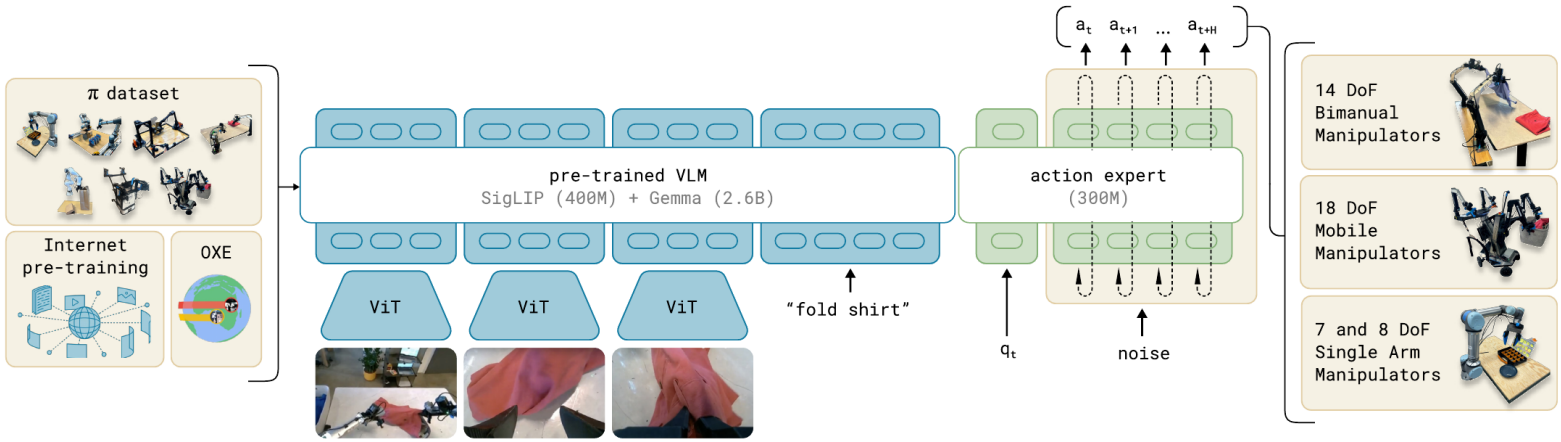


Figure 3 of Black et al., π_0, 2024.

In: three camera images, one sentence, the joint angles.

Architecture

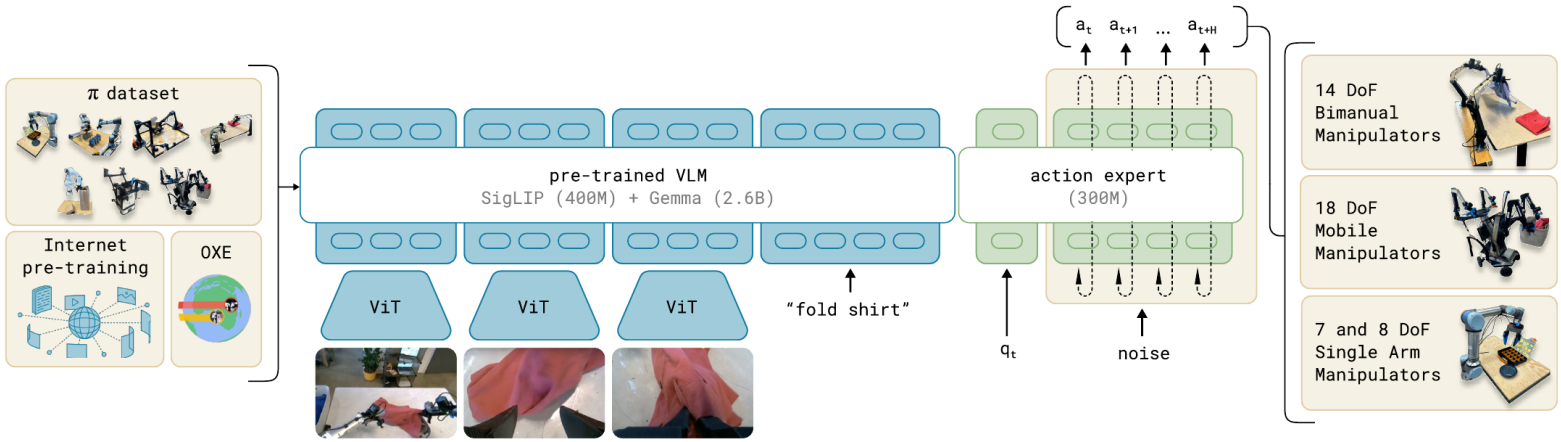


Figure 3 of Black et al., π_0, 2024.

In: three camera images, one sentence, the joint angles.

Middle: a pretrained vision-language model reads all of it, **frozen**.

Architecture

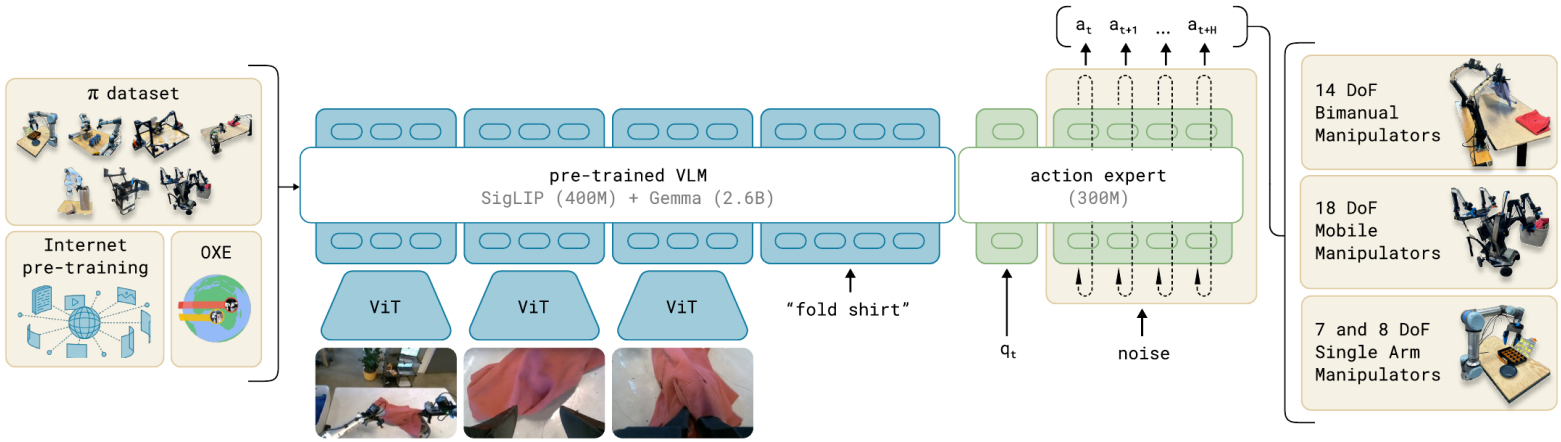


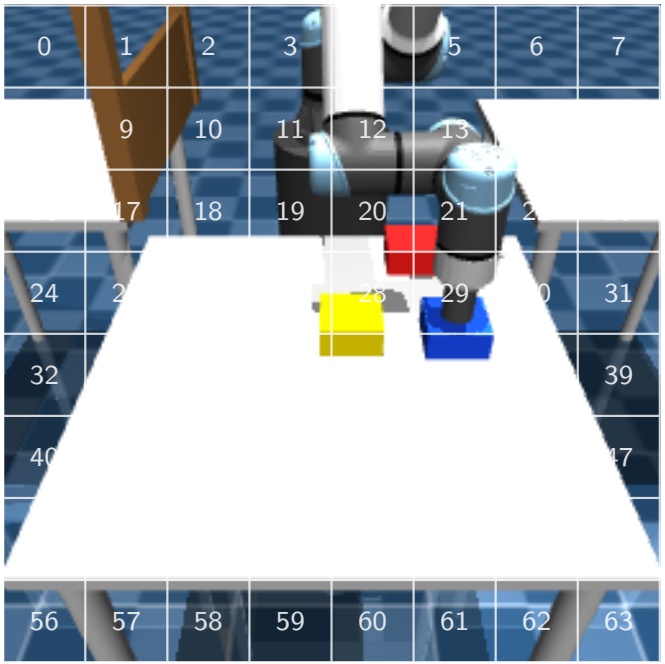
Figure 3 of Black et al., π_0, 2024.

In: three camera images, one sentence, the joint angles.

Middle: a pretrained vision-language model reads all of it, **frozen**.

Out: the action expert, trained on our demos, writes the next 50 joint commands.

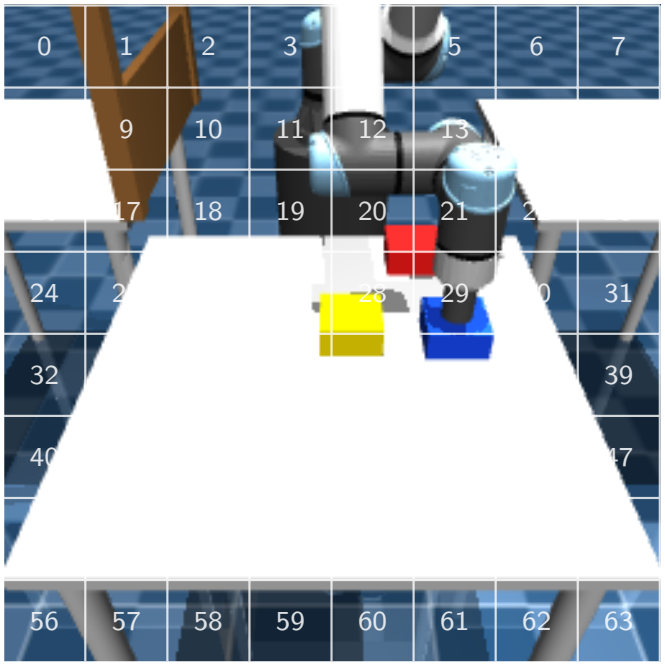
What the model sees



the front camera, 256 px, cut into 64 cells of 32 px: one token each

- **3 cameras** $\times$ 64 cells $=$ 192 image tokens

What the model sees

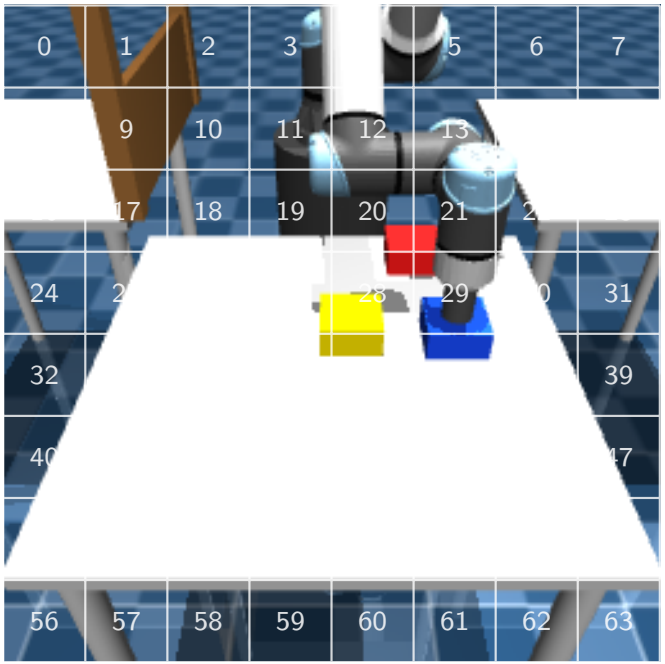


the front camera, 256 px, cut into 64 cells of 32 px: one token each

- **3 cameras** $\times$ 64 cells $=$ 192 image tokens
- **the sentence:** `place` · `the` · `blue` · `block` · `onto` · `the` · `red` · `block`

  8 word tokens, padded to 48

What the model sees

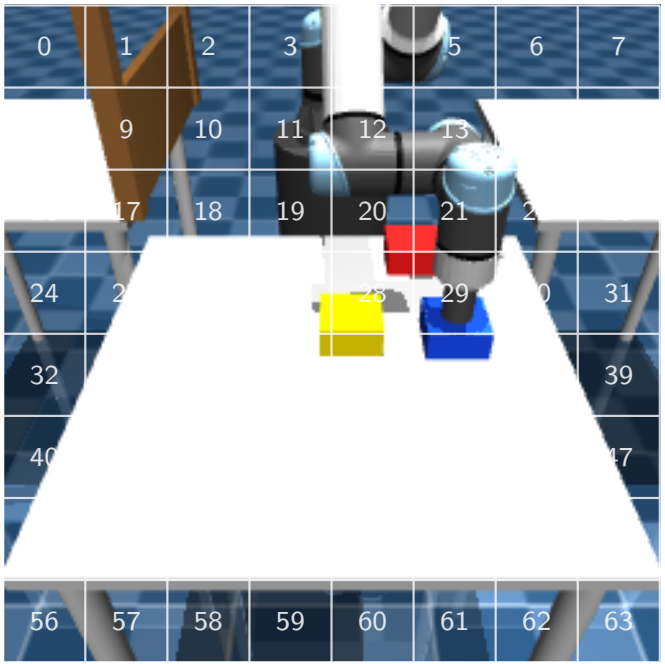


the front camera, 256 px, cut into 64 cells of 32 px: one token each

- **3 cameras** $\times$ 64 cells $=$ 192 image tokens
- **the sentence:** `place` · `the` · `blue` · `block` · `onto` · `the` · `red` · `block`

  8 word tokens, padded to 48
- **the joint angles:** 1 token

What the model sees

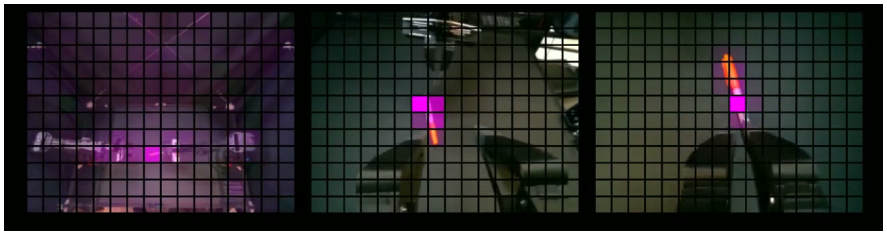


the front camera, 256 px, cut into 64 cells of 32 px: one token each

- **3 cameras** $\times$ 64 cells $=$ 192 image tokens
- **the sentence:** `place` · `the` · `blue` · `block` · `onto` · `the` · `red` · `block`
  8 word tokens, padded to 48
- **the joint angles:** 1 token

241 tokens, all the same shape. The network no longer knows which came from a pixel and which from a word. That is the trick: "blue" and the blue pixels can meet.

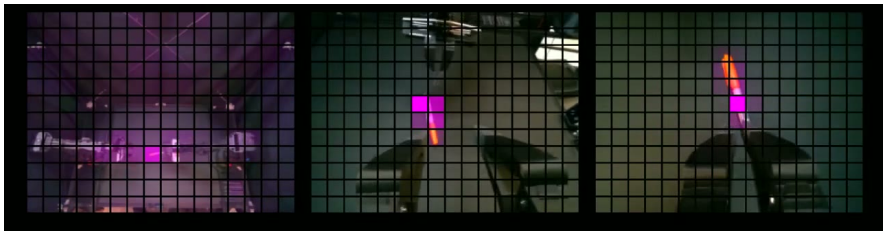
Attention: every action asks the scene what it depends on



π_0, "uncap the pen": the cells one action token weights most, across the three cameras. Welch Labs, 2025.

1. each of the 50 action tokens asks every one of the 241 inputs: *how much do I depend on you?*

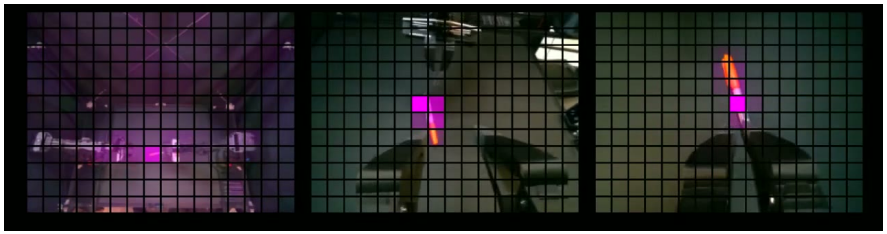
Attention: every action asks the scene what it depends on



π_0, "uncap the pen": the cells one action token weights most, across the three cameras. Welch Labs, 2025.

1. each of the 50 action tokens asks every one of the 241 inputs: *how much do I depend on you?*
2. the answers are weights that sum to one

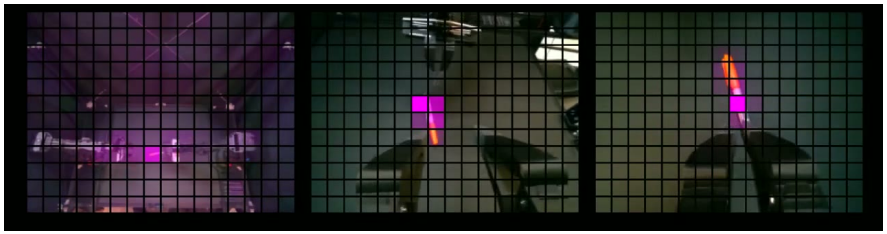
Attention: every action asks the scene what it depends on



π_0, "uncap the pen": the cells one action token weights most, across the three cameras. Welch Labs, 2025.

1. each of the 50 action tokens asks every one of the 241 inputs: *how much do I depend on you?*
2. the answers are weights that sum to one
3. the action token is rebuilt as that weighted sum, layer after layer

Attention: every action asks the scene what it depends on

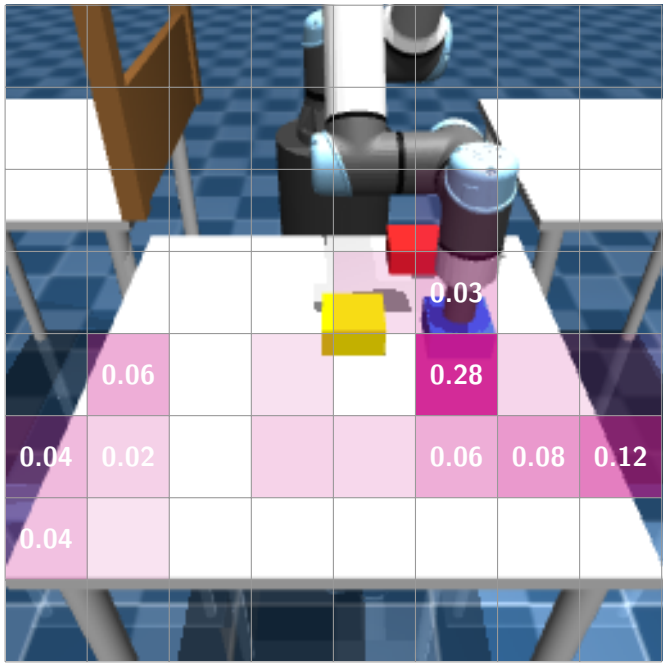


π_0, "uncap the pen": the cells one action token weights most, across the three cameras. Welch Labs, 2025.

1. each of the 50 action tokens asks every one of the 241 inputs: *how much do I depend on you?*
2. the answers are weights that sum to one
3. the action token is rebuilt as that weighted sum, layer after layer

Nobody wrote "look at the pen". The weights are learned from demonstrations, and we can read them back.

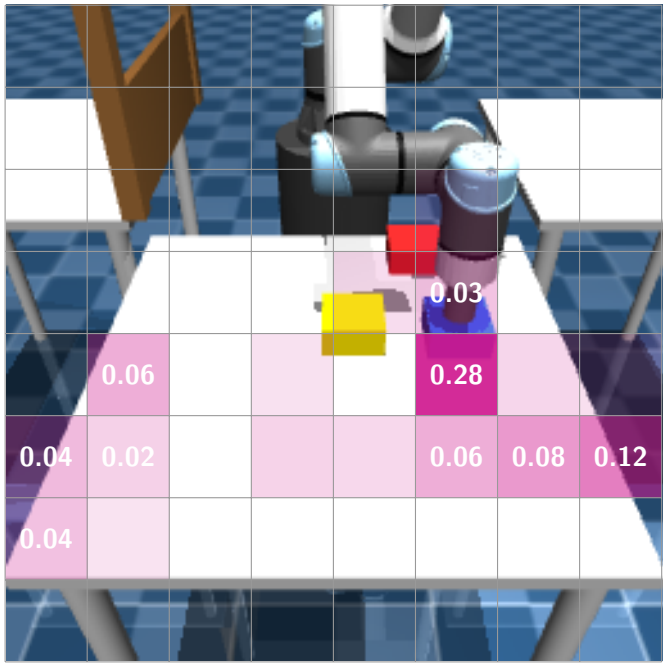
Pixels to a decision, on our robot



front camera, just before the grasp: what the first action token weights, one head

- the instruction says *blue onto red*; **0.41** of this head's weight sits on the blue block, chance is 0.03

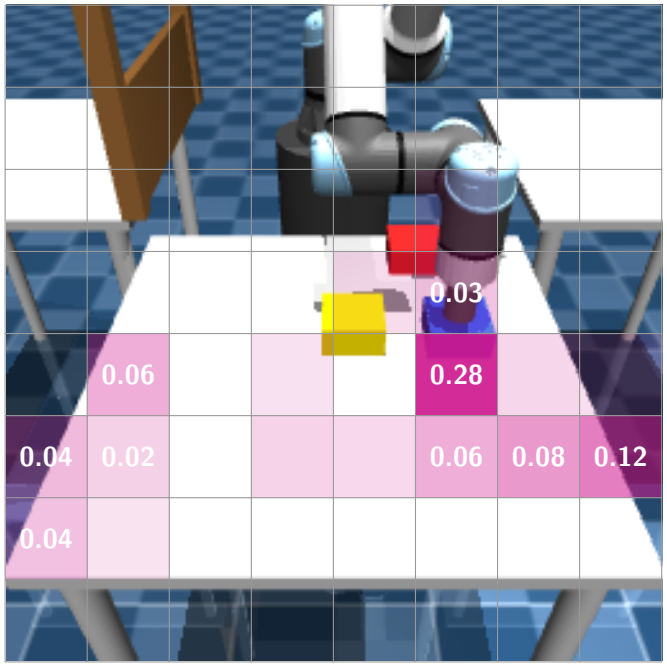
Pixels to a decision, on our robot



front camera, just before the grasp: what the first action token weights, one head

- the instruction says *blue onto red*; **0.41** of this head's weight sits on the blue block, chance is 0.03
- from that weighted sum the expert writes 50 steps $\times$ 7 numbers

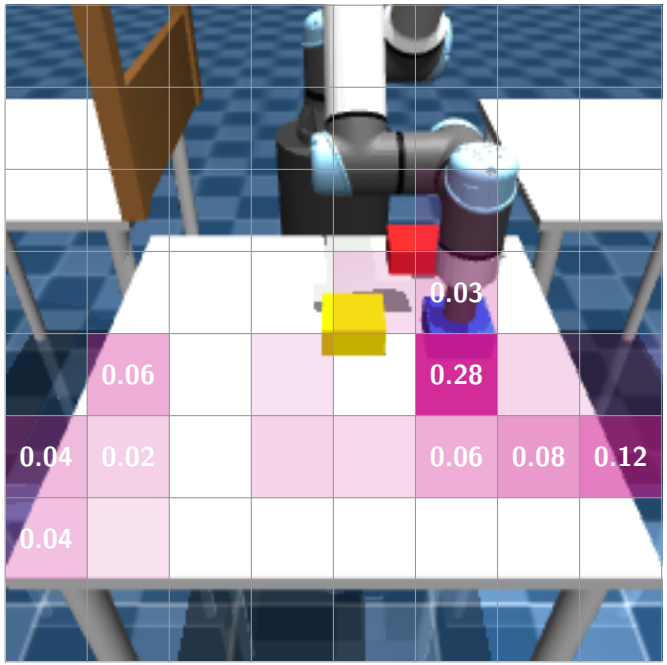
Pixels to a decision, on our robot



front camera, just before the grasp: what the first action token weights, one head

- the instruction says *blue onto red*; **0.41** of this head's weight sits on the blue block, chance is 0.03
- from that weighted sum the expert writes 50 steps $\times$ 7 numbers
- it does not write them in one go: the chunk starts as noise and is cleaned up in 10 steps, so it can hold several right answers (Part 1 of the notebook)

Pixels to a decision, on our robot



front camera, just before the grasp: what the first action token weights, one head

- the instruction says *blue onto red*; **0.41** of this head's weight sits on the blue block, chance is 0.03
- from that weighted sum the expert writes 50 steps $\times$ 7 numbers
- it does not write them in one go: the chunk starts as noise and is cleaned up in 10 steps, so it can hold several right answers (Part 1 of the notebook)

Part 2 of the notebook runs this live, then swaps one word of the instruction and watches the weights move.

The open debate: your vote

"Scale is enough"

More demos, more robots, bigger VLAs: the recipe that worked for text will work for motion.

Physical Intelligence, Google DeepMind, NVIDIA.

"You need a world model"

A policy that cannot predict consequences cannot plan, cannot be verified, and relearns physics per task.

LeCun; Dreamer, V-JEPA 2.

References

- Ahn et al., *Do As I Can, Not As I Say* (SayCan), 2022; Kambhampati et al., *LLM-Modulo*, 2024
- Chi et al., *Diffusion Policy*, RSS 2023
- Black et al., π_0*: A Vision-Language-Action Flow Model*, 2024
- Shukor et al., *SmolVLA*, 2025 (LeRobot 0.6.1, `lerobot/smolvla_base`)
- Hafner et al., *Dreamer* (2020–2025); Assran et al., *V-JEPA 2*, 2025
- Welch Labs, *How* π_0 *works*, 2025 (video)